

ZENITH

Technical Architecture & Implementation Guide

Deep-Dive Technical Documentation

Version 1.0 — May 2026

Developer: A. Prithiviraj

Table of Contents

1. Backend Architecture Deep-Dive
2. Frontend Architecture Deep-Dive
3. Authentication & Authorization Implementation
4. Storage Optimizer — Algorithm Walkthrough
5. Celery Task Pipeline
6. WebSocket Real-Time System
7. Multi-Cloud Integration Layer
8. Data Models & Validation
9. Environment Configuration
10. Architecture Decisions Log
11. Deployment & Operations

1. Backend Architecture Deep-Dive

The backend is built with FastAPI, a modern Python ASGI framework. The application entry point is **app/main.py** which creates the FastAPI instance titled "Zenith API" and mounts 7 router modules.

1.1 Router Mounting

```
app = FastAPI(title="Zenith API")
app.include_router(routes_auth.router, prefix="/api/auth")
app.include_router(routes_users.router, prefix="/api/users")
app.include_router(routes_dashboard.router, prefix="/api/dashboard")
app.include_router(routes_storage.router, prefix="/api/storage")
app.include_router(routes_security.router, prefix="/api/security")
app.include_router(routes_2fa.router, prefix="/api/2fa")
app.include_router(routes_ws.router, prefix="/ws")
```

1.2 Module Organization

Each backend module follows the Controller → Service → DB pattern, though the storage and security modules call services directly for efficiency.

Module	Files	Status
auth/	routes_auth.py, auth_service.py, auth_utils.py	Fully Implemented
users/	routes_users.py, user_model.py	Implemented
storage/	routes, optimizer, uploader, manager, tasks, tiering	Fully Implemented
security/	routes_security.py, routes_2fa.py	Fully Implemented
websockets/	routes_ws.py, connection_manager.py	Fully Implemented
dashboard/	routes_dashboard.py	Mock Data
database/	mongo_client.py	Implemented

1.3 CORS Configuration

CORS middleware is configured to allow all origins during development. In production, this should be restricted to the frontend origin only.

2. Frontend Architecture Deep-Dive

The frontend is a React 19 Single Page Application built with Vite 7. It uses createBrowserRouter for routing and React Context for state management.

2.1 Routing Configuration

Path	Component	Protection
/	HomePage	Public
/login	LoginPage	Public
/register	RegisterPage	Public
/dashboard	DashboardPage	ProtectedRoute → DashboardLayout
/dashboard/storage	StoragePage	ProtectedRoute → DashboardLayout
/dashboard/security	SecurityPage	ProtectedRoute → DashboardLayout

2.2 Component Hierarchy

- App.jsx** → Root layout rendering <Outlet />
- AuthContext.jsx** → Authentication provider (token in localStorage)
- ProtectedRoute.jsx** → Auth guard redirecting to /login
- DashboardLayout.jsx** → Sidebar + Header + content area
- Sidebar.jsx** → 5 nav items (Overview, Costs, Storage, Compute, Security)

2.3 API Layer

The centralized API service in **api.js** (239 lines) uses axios with base URL configuration. However, StoragePage.jsx and SecurityPage.jsx currently bypass this and use raw fetch() calls with their own inline API functions — this is documented technical debt.

3. Authentication & Authorization

3.1 JWT Token Flow

The system uses stateless JWT authentication with the following flow:

```
# Registration
POST /api/auth/register
Body: {"username": "...", "email": "...", "password": "..."}
→ Password hashed with bcrypt → stored in MongoDB

# Login
POST /api/auth/token (OAuth2PasswordRequestForm)
→ Verify password → Create JWT {sub: username, exp: 30min}
→ Reset 2FA verified flag → Return access_token
```

3.2 Token Validation

The `get_current_user()` dependency function decodes the JWT on every protected request, extracts the username from the "sub" claim, and looks up the user in MongoDB. For WebSocket connections, `get_current_user_ws()` reads the token from a query parameter instead of the Authorization header.

3.3 2FA Enforcement

The `require_2fa()` dependency chains on top of `get_current_user()`. It checks if the user has 2FA enabled and whether they have verified for the current session. Security endpoints use this as their auth dependency.

Endpoint	Action	Flow
POST /api/2fa/enable-2fa	Generate secret + QR	Creates TOTP secret, returns QR data URL
POST /api/2fa/finalize-2fa	First verification	Verifies code, sets two_fa_enabled=True
POST /api/2fa/verify-2fa	Session verify	Sets two_fa_verified=True for session
POST /api/2fa/disable-2fa	Remove 2FA	Unsets all 2FA fields from user doc

4. Storage Optimizer — Algorithm Walkthrough

The optimizer in `storage/optimizer.py` implements a multi-factor scoring algorithm for initial file placement recommendations.

4.1 Algorithm Steps

- Step 1:** Calculate placement score based on user priority, intent, filename analysis, file type, and size.
- Step 2:** Classify into hot/warm/cold tier based on score thresholds.
- Step 3:** Compare all CSP options for the determined tier.
- Step 4:** Select cheapest effective cost (price + retrieval penalty + duration penalty).
- Step 5:** Return recommendation + all alternatives for user override.

4.2 File Type Detection

```
def get_file_type(filename):
    if filename.endswith(('.zip', '.tar', '.gz', '.bak')): return "archive"
    if filename.endswith(('.log', '.csv', '.sql', '.json')): return "data"
    if filename.endswith(('.jpg', '.jpeg', '.png', '.mp4')): return "media"
    return "document"
```

4.3 CSP Pricing Matrix

Tier	AWS Service	\$/GB	GCP Service	\$/GB	Azure Service	\$/GB
Hot	S3 Standard	0.023	Standard	0.020	Hot Blob	0.018
Warm	S3 Standard-IA	0.0125	Nearline	0.010	Cool Blob	0.010
Cold	Glacier Flexible	0.004	Archive	0.0012	Archive	0.00099

5. Celery Task Pipeline

5.1 Worker Configuration

```
celery_app = Celery("tasks",
    broker=settings.CELERY_BROKER_URL,
    backend="rpc://",
    include=["app.storage.tasks", "app.storage.tiering_tasks"])

celery_app.conf.beat_schedule = {
    'run-storage-optimization-nightly': {
        'task': 'app.storage.tiering_tasks.run_storage_optimization',
        'schedule': crontab(hour=0, minute=0),
    },
}

celery_app.conf.timezone = 'UTC'
```

5.2 Secure File Processing Task

The `process_secure_file` task creates isolated MongoDB and S3 clients inside the task (fork safety). It scans text files for credit card patterns (regex: 13-16 digit sequences) and secret keywords (password, key, token, etc.). If sensitive content is found or manual encryption is requested, it applies AES-256 SSE and replicates to the backup bucket.

5.3 Storage Optimization Task

The nightly `run_storage_optimization` task handles bidirectional tier management:

Direction	Rule	Trigger Condition
Hot → Warm	Demotion	No access for 30+ days
Warm → Cold	Demotion	No access for 90+ days
Cold → Warm	Promotion	>2 accesses in last 5 days
Warm → Hot	Promotion	>2 accesses in last 5 days

Promotions are strictly one-level-up. Frequency scores are reset after promotion to re-evaluate access patterns.

6. WebSocket Real-Time System

The WebSocket system provides real-time push notifications for background job completion.

6.1 Connection Manager

```
class ConnectionManager:
    def __init__(self):
        self.active_connections: dict[str, WebSocket] = {}

    async def connect(self, websocket, user_id):
        await websocket.accept()
        self.active_connections[user_id] = websocket

    async def send_personal_message(self, message, user_id):
        if user_id in self.active_connections:
            await self.active_connections[user_id].send_text(message)
```

6.2 Notification Bridge

Celery tasks cannot directly access WebSocket connections. Instead, they call the internal **POST /ws/notify/{user_id}** endpoint via HTTP, which then forwards the message through the WebSocket connection manager. The frontend SecurityPage listens for "job_complete" messages and automatically refreshes the file list.

7. Multi-Cloud Integration Layer

7.1 Upload Functions

Each CSP has a dedicated upload function that handles storage class mapping:

CSP	SDK	Storage Class Mapping
AWS	boto3 (S3)	S3 Standard → STANDARD, S3 Standard-IA → STANDARD_IA, Glacier → GLACIER
GCP	google-cloud-storage	Standard → STANDARD, Nearline → NEARLINE, Archive → ARCHIVE
Azure	azure-storage-blob	Hot → StandardBlobTier.Hot, Cool → .Cool, Archive → .Archive

7.2 Manager Functions

The storage manager provides three operations per CSP: **delete** (remove object), **get_download_url** (pre-signed URL generation), and **change_tier** (storage class transition). AWS tier changes use copy-object-over-self pattern. GCP uses `update_storage_class()`. Azure uses `set_standard_blob_tier()`.

8. Data Models & Validation

8.1 Pydantic Models

```
class User(BaseModel):
    username: str = Field(..., min_length=3, max_length=50)
    email: EmailStr
    hashed_password: str

class UserInDB(User):
    two_fa_secret: Optional[str] = None
    two_fa_enabled: bool = Field(False)
    two_fa_verified: bool = Field(False)

class FileMetadata(BaseModel):
    filename: str
    s3_key: str
    owner_username: str
    size_bytes: int
    upload_date: datetime = Field(default_factory=datetime.utcnow)
    csp: str = "AWS"
    storage_class: str = "Standard"
    last_accessed_at: Optional[datetime] = None
    access_frequency_score: int = 0
    is_sensitive: bool = False
    is_encrypted: bool = False
```

9. Environment Configuration

All environment variables are managed through **pydantic-settings** BaseSettings class in `utils/config.py`, loaded from `backend/.env` via `python-dotenv`.

Variable	Purpose
MONGO_CONNECTION_STRING	MongoDB Atlas connection URI
SECRET_KEY	JWT signing key (HS256)
ALGORITHM	JWT algorithm (HS256)
ACCESS_TOKEN_EXPIRE_MINUTES	JWT expiry (30 min)
AWS_ACCESS_KEY_ID	AWS IAM access key
AWS_SECRET_ACCESS_KEY	AWS IAM secret key
S3_BUCKET_NAME	Standard storage bucket
SECURE_S3_BUCKET_NAME	Secure files primary bucket
REPLICA_S3_BUCKET_NAME	Secure files backup bucket
REPLICA_S3_REGION	Replica region (us-east-1)
CELERY_BROKER_URL	CloudAMQP RabbitMQ URL
GCP_SERVICE_ACCOUNT_JSON_PATH	GCP service account key path
GCP_BUCKET_NAME	GCP storage bucket
AZURE_STORAGE_ACCOUNT_NAME	Azure storage account
AZURE_STORAGE_ACCOUNT_KEY	Azure account key
AZURE_CONTAINER_NAME	Azure blob container

10. Architecture Decisions Log

ID	Decision	Date	Rationale
DEC-001	Monorepo (React + FastAPI)	2025-08	Separate stacks, shared repo
DEC-002	MongoDB Atlas	2025-08	Flexible schema for multi-cloud metadata
DEC-003	JWT Stateless Auth	2025-08	Scalable, no server sessions
DEC-004	Celery + CloudAMQP	2025-09	Non-blocking file processing
DEC-005	WebSocket Notifications	2025-09	Real-time task completion updates
DEC-006	Multi-Cloud (AWS+GCP+Azure)	2025-09	Cross-provider cost optimization
DEC-007	Scoring-Based Placement	2025-09	Intelligent tier recommendation
DEC-008	TOTP 2FA on Security	2025-10	Extra protection for sensitive files
DEC-009	Separate Collections	2025-10	Different access controls per type
DEC-010	Three S3 Buckets	2025-10	Standard + secure + DR replica
DEC-011	Branded as "Zenith"	2025-08	Product identity across all surfaces

11. Deployment & Operations

11.1 Local Development Setup

Prerequisites: Python 3.x, Node.js, MongoDB Atlas account, AWS/GCP/Azure credentials, CloudAMQP account

```
# Terminal 1: Backend API
cd backend && source ../venv/bin/activate
uvicorn app.main:app --reload # → http://localhost:8000

# Terminal 2: Frontend
cd frontend && npm run dev # → http://localhost:5173

# Terminal 3: Celery Worker
cd backend && source ../venv/bin/activate
celery -A app.celery_worker worker --loglevel=info

# Terminal 4: Celery Beat Scheduler
cd backend && source ../venv/bin/activate
celery -A app.celery_worker beat --loglevel=info
```

11.2 Infrastructure as Code

Terraform configurations in `devops/aws/` define S3 buckets, IAM policies, DynamoDB tables, and Lambda roles. Docker configurations provide separate Dockerfiles for frontend and backend with docker-compose orchestration. CI/CD pipelines are configured for both GitHub Actions and GitLab CI.

— End of Technical Architecture Document —